GUÍA DE TRABAJO PARA EL ESTUDIANTE
SEMANAS 5 Y 6

Colecciones Dinámicas (ArrayList) y Traducción de Relaciones 1..* de UML a Java

Sección 1: Contextualización y Reto de la Comercializadora

La Tienda Amazonica de la **Universidad de la Amazonia** requiere sistematizar su punto de venta de productos regionales (pulpas de arazá, copoazú y queso caqueteño). En la realidad organizacional, las ventas son impredecibles: un cliente puede adquirir un solo producto o múltiples en una misma compra.

Cuando el número de elementos varía durante la ejecución, el uso de arreglos estáticos de tamaño fijo puede resultar ***rígido*** o requerir una gestión manual compleja. Por esta razón, en esta guía exploraremos la colección **ArrayList** de la API estándar de Java, como una alternativa adecuada para gestionar colecciones cuyos elementos cambian dinámicamente en tiempo de ejecución.

Sección 2: Arreglos Tradicionales vs. Colecciones Dinámicas (ArrayList)

Para comprender cuándo y por qué utilizar ArrayList frente a un arreglo tradicional, es indispensable analizar sus características técnicas sin asumir que una estructura reemplaza universalmente a la otra, sino que responden a necesidades de diseño distintas:

Criterio de Comparación	Arreglos Tradicionales (Tipo[])	Colecciones Dinámicas (ArrayList<Tipo>)
Naturaleza del Tamaño	Estructura de tamaño fijo definido al instanciar.	Estructura dinámicamente apropiada para colecciones de tamaño variable.
Sintaxis de Declaración	private DetalleFactura[] detalles;	private ArrayList<DetalleFactura> detalles;
Inicialización y Referencia	private DetalleFactura[] d = new DetalleFactura[5]; (Crea referencia y arreglo de 5 casillas)	private ArrayList<DetalleFactura> d; (Solo crea la referencia. Requiere new ArrayList<>() para instanciar en memoria).
Medición de Elementos	.length retorna el número fijo de posiciones del arreglo.	.size() retorna el número actual de elementos almacenados.

Acceso a Posiciones	Sintaxis de corchetes directos: detalles[i].	Mediante métodos definidos por la clase ArrayList: detalles.get(i).
Inserción y Eliminación	Manual por índice; requiere reordenamiento explícito de casillas.	Mediante métodos operacionales: .add(objeto) y .remove(indice).
Soporte de Tipos	Soporta tipos primitivos (int[]) y tipos referencia.	Trabaja con tipos referencia; los tipos primitivos se representan mediante sus clases envolventes (autoboxing).
Gestión de Excepciones	Puede lanzar ArrayIndexOutOfBoundsException.	Puede lanzar IndexOutOfBoundsException si se accede a un índice fuera de rango.

🔍 Precisión Arquitectónica: Multiplicidad 1..* vs. Instanciación en Código

En el modelo UML, la relación Factura ── (1..*) DetalleFactura expresa una regla de negocio: una factura debe incorporar al menos un detalle para considerarse válida en el dominio del problema. Sin embargo, al ejecutar new ArrayList<DetalleFactura>() en el constructor de Java, la colección se crea inicialmente con 0 elementos en memoria. Es importante comprender que la declaración inicial del ArrayList en código permite almacenar de 0 a muchos elementos (0..*), y es la lógica del sistema (métodos de validación) la que garantiza que la factura no se procese definitivamente sin cumplir la regla 1..* del negocio.

Sección 3: Catálogo de Métodos Esenciales de ArrayList

La clase ArrayList del paquete java.util ofrece métodos públicos para operar sobre la colección de objetos:

- **public boolean add(E elemento)**: Añade el objeto al final de la colección de forma dinámica.
- **public void add(int indice, E elemento)**: Inserta un objeto en una posición específica, desplazando los elementos posteriores.
- **public E get(int indice)**: Recupera la referencia del objeto ubicado en el índice especificado (iniciando en 0). Lanzará IndexOutOfBoundsException si el índice no es válido.
- **public E remove(int indice)**: Elimina la referencia en la posición indicada y reajusta las posiciones posteriores.
- **public int size()**: Retorna un entero con el número actual de elementos almacenados en la lista.
- **public boolean isEmpty()**: Retorna true si la colección no contiene elementos (size() == 0) y false en caso contrario.

Sección 3.1: Comprobación Conceptual previa a la Programación
Antes de escribir el código fuente en Java, responde conceptualmente a las siguientes preguntas clave para asegurar la correcta transición desde el modelo UML:

1. **¿Qué clase del modelo actúa como contenedora y alberga la colección?**
2. **¿Qué clase representa cada elemento individual almacenado en dicha colección?**
3. **¿Qué operación o método de ArrayList permite registrar un nuevo elemento en la colección?**
4. **¿Qué método permite consultar el número actual de elementos almacenados para controlar los recorridos?**

Sección 4: Implementación Completa en Java (Tienda Amazonica)
A continuación se presenta la implementación en Java de las cuatro clases del dominio. Observa cómo las asociaciones simples (1 a 1) se declaran como variables de objeto directas, mientras que la colección 1..* se implementa mediante ArrayList.

Clase: Cliente.java (Asociación Simple)

```
package uniamazonia.tiendaamazonica.mundo;
/**
 * Modela al comprador en la tienda Amazonica institucional.
 */
public class Cliente {
    private String cedula;
    private String nombre;
    private String correo;

    public Cliente(String cedula, String nombre, String correo) {
        this.cedula = cedula;
        this.nombre = nombre;
        this.correo = correo;
    }

    public String darCedula() {
        return this.cedula;
    }

    public String darNombre() {
        return this.nombre;
    }
```

```
}
```

Clase: Producto.java (Asociación Simple)

```
package uniamazonia.tiendaamazonica.mundo;

/**
 * Modela los artículos agroecológicos disponibles en el catálogo.
 */
public class Producto {
    private String codigo;
    private String nombre;
    private double precioUnitario;

    public Producto(String codigo, String nombre, double precioUnitario) {
        this.codigo = codigo;
        this.nombre = nombre;
        this.precioUnitario = precioUnitario;
    }

    public double darPrecioUnitario() {
        return this.precioUnitario;
    }
}
```

Clase: DetalleFactura.java (Asociación 1 a 1 con Producto)

```
package uniamazonia.tiendaamazonica.mundo;

/**
 * Representa cada renglón o línea individual de compra.
 */
public class DetalleFactura {
    private int cantidad;
    private double subtotal;

    // Asociación simple 1 a 1 hacia Producto (Atributo de tipo referencia)
    private Producto producto;

    public DetalleFactura(int cantidad, Producto producto) {
        this.cantidad = cantidad;
```

```
        this.producto = producto;
        this.subtotal = calcularSubtotal();
    }

    public double calcularSubtotal() {
        if (this.producto != null) {
            this.subtotal = this.cantidad * this.producto.darPrecioUnitario();
        } else {
            this.subtotal = 0.0;
        }
        return this.subtotal;
    }
}
```

Clase: Factura.java (Asociación 1 a 1 con Cliente y Colección con DetalleFactura)

```
package uniamazonia.tiendaamazonica.mundo;

import java.util.ArrayList; // Importación de la colección de Java

/**
 * Representa el comprobante de venta o contenedor principal.
 */
public class Factura {
    private int numeroFactura;
    private String fecha;
    private double total;

    // Asociación simple 1 a 1 hacia Cliente
    private Cliente cliente;

    // Declaración de la referencia a la colección (no crea el objeto en memoria)
    private ArrayList<DetalleFactura> detalles;

    public Factura(int numeroFactura, String fecha, Cliente cliente) {
        this.numeroFactura = numeroFactura;
        this.fecha = fecha;
        this.cliente = cliente;

        // Instanciación explícita del objeto ArrayList en memoria dentro del constructor
        this.detalles = new ArrayList<DetalleFactura>();
        this.total = 0.0;
```

```
    }

    // Método operacional que utiliza .add() de ArrayList
    public void agregarDetalle(DetalleFactura detalle) {
        if (detalle != null) {
            this.detalles.add(detalle);
        }
    }

    // Método de acumulación que recorre la colección
    public double calcularTotal() {
        double acumulado = 0.0;
        for (int i = 0; i < this.detalles.size(); i++) {
            DetalleFactura det = this.detalles.get(i);
            acumulado += det.calcularSubtotal();
        }
        this.total = acumulado;
        return this.total;
    }
}
```

🔗 **Cadena Conceptual de Colaboración en calcularTotal():**

Es fundamental observar cómo en el método calcularTotal() de Factura se despliega la siguiente cadena conceptual de colaboración entre objetos:
Factura ⟶ ArrayList ⟶ DetalleFactura ⟶ Producto ⟶ darPrecioUnitario()

Esta secuencia demuestra que ArrayList no reemplaza las relaciones de objetos entre clases, sino que actúa como la estructura que contiene a las instancias de DetalleFactura, las cuales a su vez colaboran con la clase Producto para obtener la información necesaria.

Sección 5: Laboratorio con Inteligencia Artificial.
En este laboratorio utilizaremos NotebookLM como un Auditor Socrático para validar nuestros diseños frente a las fuentes oficiales de la asignatura.

🛠️ **Configuración Inicial en NotebookLM:**
1. Inicia sesión en NotebookLM con tu correo institucional (@uniamazonia.edu.co).
2. Crea un cuaderno denominado: 'Arquitectura Tienda Amazónica'.
3. Carga como fuentes: esta Guía de Trabajo, el texto de Jorge Villalobos y el libro de Luis Joyanes Aguilar.

🎙️ Paso Cero: Debate en Audio (Audio Overview):

En el panel de Estudio de NotebookLM, genera un Resumen en Audio (Audio Overview). El audio debe abordar un debate sobre por qué ArrayList puede ser una alternativa adecuada frente a un arreglo tradicional cuando la cantidad de elementos varía durante la ejecución en un sistema de facturación.

Prompts de Auditoría.

📌 Prompt 1: Auditoría UML.

> *"Actúa como un expert en Desarrollo de Software con basta (15 años) experiencia en recoleccion de requerimientos, analisis y diseño de sistemas de información y audita mi modelo. Revisa si en la relación entre Factura y DetalleFactura cometí el error de escribir el atributo de la asociación dentro del compartimento de atributos de la clase Factura cuando ya existe una línea de asociación dibujada. Si identificas este problema, explícame por qué ocurre según la convención de modelado adoptada en esta guía y propone la corrección."*

📌 Prompt 2: Auditoría Java en Factura.java (Importación e Instanciación)

> *"Actúa como mi Asistente Experto de Lógica II. Revisa mi clase Factura.java y audita dos condiciones:*
> *1. ¿Se importó java.util.ArrayList?*
> *2. ¿Se instanció explícitamente el objeto ArrayList dentro del constructor mediante this.detalles = new ArrayList<DetalleFactura>();?*
> *Si omití la instanciación, identifica el problema, explícame por qué se produciría un NullPointerException en tiempo de ejecución al invocar agregarDetalle() y propone la corrección sintáctica."*

📌 Prompt 3: Auditoría de Bucle y Recorrido de Colección

> *"Audita el método calcularTotal() de mi clase Factura.java. Verifica si utilicé this.detalles.size() para controlar el límite superior del bucle for y this.detalles.get(i) para acceder a los elementos. Si utilicé sintaxis de corchetes [i] o .length, identifica el problema, explícame por qué esa sintaxis no corresponde a la API de ArrayList y propone la estructura correcta."*

Sección 6: Transferencia Práctica y Lista de Chequeo de Calidad

Reúnase con su equipo de proyecto integrador e identifique una relación 1..* en su problema de estudio. Implemente las clases correspondientes garantizando la correcta instanciación de la colección y su recorrido.

Lista de Chequeo de Autoevaluación:

[ ] ¿Todos los atributos están encapsulados con visibilidad private?

[ ] ¿Se importó explícitamente java.util.ArrayList al inicio del archivo?

[ ] ¿Se instanció el objeto ArrayList dentro del constructor (new ArrayList<...>()) y no solo su referencia?

[ ] ¿Se utilizaron los métodos de la API de ArrayList (.size(), .get(), .add()) en lugar de corchetes [] o .length?

[ ] ¿La multiplicidad definida en el diagrama UML coincide con el comportamiento permitido por el código fuente?